

Change or Crisis? What Framing Reveals About Climate Conversations on Bluesky

COMP3020 Social Web Analytics – Group Project

09 October 2026

Team members	Student ID	Contribution
Aakash Jadhav	22078242	34%
Miciella Ngo	22119913	33%
Pushti Patel	22147139	33%

1 Introduction

Climate change is a widely discussed topic on social media, but people do not all describe it in the same way. Some use the neutral, descriptive term “climate change”, while others use “climate crisis” to stress urgency and the need for action, and some news organisations and campaigners have moved towards this more urgent wording. This report investigates whether the choice between these two framings is linked to differences in what people discuss, how much engagement their posts receive and how they interact with each other on the social platform Bluesky.

We collected public Bluesky posts containing each phrase through the Bluesky API in R, and analysed them using techniques from COMP3020. The investigation is organised around *four related research questions*:

- **RQ1 (Hypothesis testing):** Is the level of engagement (likes) related to whether a post uses “climate crisis” or “climate change” framing?
- **RQ2 (Text analysis):** Do “climate crisis” and “climate change” posts differ in the words and topics they use?
- **RQ3 (Clustering):** Can posts be grouped into distinct themes based on their words, and are the two framings distributed differently across these themes?
- **RQ4 (Network analysis):** Which accounts occupy the most important positions in the reply network, and do the two framings differ in how much reply interaction they involve?

Each analysis builds on the previous one. The hypothesis test (RQ1) checks whether framing is related to engagement; finding no clear difference, we then examine whether framing relates to *what* is said instead (RQ2). The text analysis suggests several overlapping themes, which clustering identifies formally and links back to framing (RQ3). Finally, the network analysis examines *who* interacts with whom and whether the framings differ in conversation (RQ4).

2.1 Data Collection

Topic: We investigate how the framing of climate discussions on Bluesky differs between posts that say “climate crisis” and posts that say “climate change”.

What? We collected public Bluesky posts containing each phrase, then cleaned them so every post belongs to exactly one framing group.

Why? Bluesky is an open social platform with a free public API. Its posts include text, engagement counts (likes, reposts, replies) and mentions, which give us the data needed for all four analyses: text analysis, hypothesis testing, clustering and network analysis.

How the data was accessed and obtained

Data was collected from the Bluesky search API using the `atrrrr` R package. We logged in with a Bluesky app password, which was typed at the prompt and never stored in the code or the GitHub repository.

The Bluesky code was run once on **1 Oct 2026**. It is commented out (`#`) below because running it again would return different posts and change our results. All outputs in this report come from the saved data files.

```
# Bluesky login and search function (run once on 1 Oct 2026, commented out)

# library(atrrrr) # connecting to Bluesky and collecting posts
#
# # Log in to Bluesky (app password typed at the prompt, never saved in code)
# auth(user = "aakash18.bsky.social")
#
# # Search function that retries up to 3 times if Bluesky gives a server error (e.g. 502)
# get_posts <- function(t, limit = 500, tries = 3) {
#   for (i in 1:tries) {
#     p <- tryCatch(search_post(t, limit = limit), error = function(e) NULL)
#     if (!is.null(p)) return(p)
#     message("Retry ", i, " for: ", t)
#     Sys.sleep(5)
#   }
#   NULL
# }
```

The search function retries automatically because the Bluesky API occasionally returned “502 Bad Gateway” server errors during collection.

R Packages used for analysis and visualisation

`atrrrr` was used only for data collection (the commented-out code above), so it is not loaded here.

```
library(dplyr)      # working with tables (filtering, grouping, counting)
library(stringr)    # working with text (detecting phrases, counting words)
library(knitr)       # displaying tables in the report
library(ggplot2)     # plots
library(tm)          # text mining
library(SnowballC)   # stemming
library(wordcloud)   # word clouds
library(RColorBrewer) # colour palettes
library(igraph)      # network analysis
library(kableExtra)  # fitting wide tables to the page width after converting it in to PDF format
```

Choosing the topic (pilot)

Before committing to a topic, we collected a sample of about 550–600 posts for each of six candidate topics. We compared them on what our analyses need: enough posts, many different authors (for the network), engagement (for the hypothesis test) and mentions and replies (for the network).

```
# Pilot code (run once on 1 Oct 2026, commented out)

# topics <- c("nursing shortage", "AI healthcare", "housing crisis australia",
#            "climate change", "climate crisis", "net zero")
#
# # For each topic: collect posts and summarise volume, authors, engagement and mentions
# pilot <- lapply(topics, function(t) {
```

```
# p <- get_posts(t)
# Sys.sleep(2)
# if (is.null(p)) return(tibble(topic = t, posts = NA))
# tibble(
#   topic          = t,
#   posts          = nrow(p),
#   unique_authors = n_distinct(p$author_handle),
#   pct_liked      = round(mean(p$like_count > 0) * 100, 1),
#   median_likes   = median(p$like_count),
#   pct_mentions  = round(mean(str_detect(p$text, "@\\w")) * 100, 1),
#   pct_replied    = round(mean(p$reply_count > 0) * 100, 1)
# )
# }) /> bind_rows()
#
# saveRDS(pilot, "pilot.rds")
```

```
# Load and display the saved pilot results
pilot <- readRDS("pilot.rds")
kable(pilot, caption = "Pilot comparison of six candidate topics")
```

Table 1: Pilot comparison of six candidate topics

topic	posts	unique_authors	pct_liked	median_likes	pct_mentions	pct_replied
nursing shortage	NA	NA	NA	NA	NA	NA
AI healthcare	595	469	31.8	0	4.5	24.0
housing crisis australia	554	359	56.9	1	4.3	29.4
climate change	588	477	50.2	1	6.0	25.0
climate crisis	595	493	59.2	1	6.2	30.6
net zero	592	424	42.9	0	5.6	34.0

“Climate crisis” and “climate change” had the most unique authors (493 and 477) and the highest mention rates (6.2% and 6.0%), which matter most for the network analysis. “Climate crisis” also had the highest share of liked posts (59.2%). “Housing crisis australia” had good engagement (56.9% liked) but the fewest authors (359), which would give a smaller network. “AI healthcare” and “net zero” had a median of 0 likes, which would make engagement comparisons weak. “Net zero” had the highest reply rate (34.0%), but its lower engagement and fewer authors made climate the stronger overall choice. “Nursing shortage” could not be collected because the API kept returning server errors. We therefore chose climate, and comparing the two phrases gave us a clear research angle: whether the framing changes the conversation.

Full data collection

A check of 100 sample “climate crisis” posts showed some non-English posts and some posts that matched the search without using the phrase in their text. We therefore searched for the **exact phrases** (using quotes) with an **English-only filter** (`lang:en`), requesting 2,000 posts for each phrase (the API returned slightly more: 2,085 and 2,063).

```
# Main collection code (run once on 1 Oct 2026, 7:49pm AEST, commented out)

## Quotes make Bluesky search for the exact phrase; lang:en keeps English posts only
# q_crisis <- get_posts('"climate crisis" lang:en', limit = 2000)
# Sys.sleep(5)
# q_change <- get_posts('"climate change" lang:en', limit = 2000)
#
## Combine both searches and label where each post came from
```

```
# posts_raw <- bind_rows(
#   q_crisis |> mutate(search_term = "climate crisis"),
#   q_change |> mutate(search_term = "climate change")
# )
#
# saveRDS(posts_raw, "posts_raw.rds")
```

Variables collected and data volume

```
# Load the saved raw data
posts_raw <- readRDS("posts_raw.rds")

# Data volume: number of posts (rows) and variables (columns)
dim(posts_raw)
```

```
## [1] 4148 22
```

The raw data contains **4,148 posts and 22 variables**: 21 returned by the API, plus the `search_term` label we added.

```
# Number of posts collected for each phrase
count(posts_raw, search_term)
```

```
## # A tibble: 2 x 2
##   search_term      n
##   <chr>         <int>
## 1 climate change 2063
## 2 climate crisis 2085
```

The two searches returned a similar number of posts: 2,085 for “climate crisis” and 2,063 for “climate change”.

```
# Preview of the collected data (first 6 posts, main columns)
head(posts_raw[, c("author_handle", "search_term", "like_count", "reply_count")])
```

```
## # A tibble: 6 x 4
##   author_handle      search_term  like_count reply_count
##   <chr>          <chr>         <int>      <int>
## 1 stevedawe.bsky.social climate crisis      0          0
## 2 rayminehane.bsky.social climate crisis      3          2
## 3 commissionerhr.coe.int climate crisis      2          1
## 4 serenaribena.bsky.social climate crisis      1          1
## 5 josiahmortimer.bsky.social climate crisis      8          0
## 6 mzungu.aus.social.ap.brid.gy climate crisis      0          0
```

The preview shows the structure of the data: each row is one post, with its author, the phrase it was found with, and its engagement counts. Engagement is low for most posts (0 to 8 likes in this sample), which we explore further in the hypothesis test (2.2).

```
# Example posts that matched the search but don't use the phrase in their text
# (these are removed during cleaning)
posts_raw |>
  filter(!str_detect(str_to_lower(text), "climate crisis|climate change")) |>
  select(search_term, text) |>
  head(3)
```

```
## # A tibble: 3 x 2
##   search_term    text
##   <chr>         <chr>
## 1 climate crisis Australia. Another home of Climate complacent Government: The ~
## 2 climate crisis Comment from Climate Resistance
## 3 climate crisis chris minns is a dog https://www.theguardian.com/commentisfree~
```

These posts were returned by the “climate crisis” search even though their text doesn’t contain the phrase. They were likely matched through a linked article or quoted post. This shows why a cleaning step was needed to make sure each post actually uses the framing we are studying.

```
# Main variables collected and how they are used
variables <- data.frame(
  Variable = c("uri", "author_handle", "text", "like_count",
               "repost_count", "reply_count", "indexed_at", "search_term"),
  Description = c("Unique ID of the post",
                  "Bluesky username of the author",
                  "Text of the post",
                  "Number of likes",
                  "Number of reposts",
                  "Number of replies",
                  "Date and time the post was indexed",
                  "Which phrase the post was found with (added by us)"),
  Used_in = c("Cleaning (duplicates)", "Network labels (2.5)", "Text analysis, clustering (2.3,
    ↪ 2.4)",
              "Hypothesis test (2.2)", "Not used in analysis", "Pilot (topic choice)",
              "Not used in analysis", "Data volume (posts per phrase)")
)
kable(variables, caption = "Main variables collected and how they are used")
```

Table 2: Main variables collected and how they are used

Variable	Description	Used_in
uri	Unique ID of the post	Cleaning (duplicates)
author_handle	Bluesky username of the author	Network labels (2.5)
text	Text of the post	Text analysis, clustering (2.3, 2.4)
like_count	Number of likes	Hypothesis test (2.2)
repost_count	Number of reposts	Not used in analysis
reply_count	Number of replies	Pilot (topic choice)
indexed_at	Date and time the post was indexed	Not used in analysis
search_term	Which phrase the post was found with (added by us)	Data volume (posts per phrase)

The other columns returned by the API include nested author, post and embed details (author_data, post_data, embed_data) and ready-made fields for replies (in_reply_to, in_reply_root), mentions, quotes, hashtags (tags), links and language (langs). These are kept in posts_raw.rds; the reply field (in_reply_to) provides the edges for the network analysis (2.5).

Cleaning the records

We cleaned the records in four steps:

1. Removed empty posts and posts that appeared in both searches.
2. Kept only posts whose text uses **exactly one** of the two phrases. Posts using neither phrase (matched only through a link or quoted post) or both phrases cannot be placed in one framing group.

3. Removed very short posts (fewer than 5 words), as they don't contain enough text to analyse.
4. Removed posts with identical text (bots and copy-paste spam).

```
# Start from the saved raw data so the result is the same every time
n0 <- nrow(posts_raw)

# Step 1: remove empty posts and duplicates, then label each post's framing
posts_clean <- posts_raw |>
  filter(!is.na(text), text != "") |>
  distinct(uri, .keep_all = TRUE) |>
  mutate(
    text_lower = str_to_lower(text),
    has_crisis = str_detect(text_lower, "climate crisis"),
    has_change = str_detect(text_lower, "climate change"),
    framing = case_when(
      has_crisis & has_change ~ "both",
      has_crisis ~ "crisis",
      has_change ~ "change",
      TRUE ~ NA_character_
    ),
    n_words = str_count(text, "\\S+")
  )
n1 <- nrow(posts_clean)

# Step 2: keep posts that use only ONE of the two phrases
posts_clean <- posts_clean |> filter(!is.na(framing), framing != "both")
n2 <- nrow(posts_clean)

# Step 3: remove very short posts (fewer than 5 words)
posts_clean <- posts_clean |> filter(n_words >= 5)
n3 <- nrow(posts_clean)

# Step 4: remove posts with identical text (bots, spam)
posts_clean <- posts_clean |> distinct(text_lower, .keep_all = TRUE)
n4 <- nrow(posts_clean)

# Number of posts left after each cleaning step
cleaning_steps <- data.frame(
  Step = c("Raw posts", "Empty/duplicate posts removed",
           "Phrase in text (one framing only)", "5+ words", "Duplicate text removed"),
  Posts = c(n0, n1, n2, n3, n4)
)
kable(cleaning_steps, caption = "Posts remaining after each cleaning step")
```

Table 3: Posts remaining after each cleaning step

Step	Posts
Raw posts	4148
Empty/duplicate posts removed	4104
Phrase in text (one framing only)	2507
5+ words	2494
Duplicate text removed	2400

Most steps removed only a small number of posts: 44 empty or duplicate posts, 13 very short posts and 94 posts with identical text. The biggest drop (4,104 to 2,507) came from removing posts that did not actually use either phrase in their text, as shown in the examples above.

```
# Final number of clean posts in each framing
kable(count(posts_clean, framing), caption = "Clean posts by framing")
```

Table 4: Clean posts by framing

framing	n
change	1513
crisis	887

After cleaning, **2,400 posts** remain: 1,513 “climate change” and 887 “climate crisis”. Only 887 crisis-framed posts remain from the 2,085 results of the “climate crisis” search, compared with 1,513 change-framed posts from the 2,063 results of the “climate change” search. This suggests the “climate crisis” search matched more posts through links and quoted content. The clean data is saved as `posts_clean.rds` and used in 2.2, 2.3 and 2.4.

Limitations - **Snapshot in time:** the search returns recent posts, so our data only reflects the conversation around 1 Oct 2026. Results could be different at another time, for example during a major climate event. - **API limits:** we requested 2,000 posts per phrase, not every post on the topic, and the API sometimes returned server errors (which is why “nursing shortage” failed in the pilot). - **Search matching:** about 1,600 posts matched the search without using the phrase in their text, so they were removed. The English filter depends on the language tag set by the user’s app, which may not always be correct. - **Unequal groups:** there are more “change” posts (1,513) than “crisis” posts (887), so we compare proportions instead of raw counts where this matters. - **Representativeness:** Bluesky users are not representative of all social media users or the general public, so our findings describe the Bluesky conversation only.

2.2 Hypothesis Testing

Research question: Is the level of engagement (likes) related to whether a post uses “climate crisis” or “climate change” framing?

What? We test whether framing and engagement are related, using a chi-squared test of independence (Module 3).

Why? If “crisis” framing sounds more urgent, it may attract more engagement than “change” framing. This test tells us whether any difference in engagement between the two framings is larger than we would expect by chance.

Hypotheses:

- **H0:** Framing and engagement level are independent.
- **H1:** Framing and engagement level are not independent.

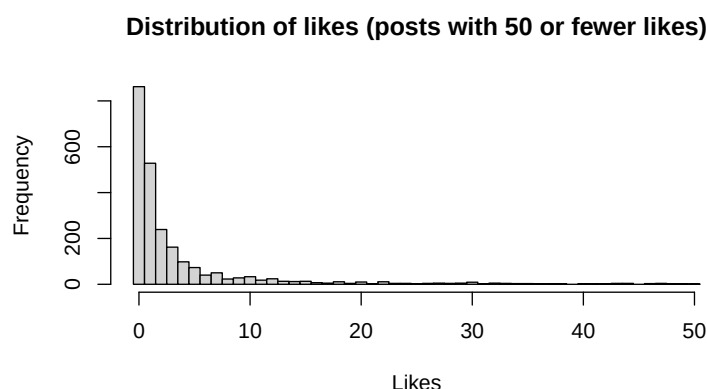
Assumptions: Likes are counts with many zeros and a long right tail, so they are not normally distributed. Instead of comparing means, we group likes into three engagement levels and compare the proportion of posts in each level. The chi-squared approximation requires most expected counts to be at least 5. We check this below, and also report a simulated p-value as shown in Module 3.

```
# Load the cleaned posts from 2.1
posts <- readRDS("posts_clean.rds")

# Check the distribution of likes
summary(posts$like_count)
```

```
##      Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##      0.00    0.00    1.00   11.36    4.00 8399.00
```

```
# Histogram of likes (limited to 0-50 likes so the shape is visible;
# a few posts have far more, up to 8,399)
# Each bar = one like count (0, 1, 2, ...)
hist(posts$like_count[posts$like_count <= 50], breaks = seq(-0.5, 50.5, by = 1),
      main = "Distribution of likes (posts with 50 or fewer likes)",
      xlab = "Likes")
```



Results -

1. Likes are heavily skewed: the median is 1 like but the mean is 11.36, with a maximum of 8,399. This confirms that comparing means would be misleading, so we group likes into engagement levels.
2. The histogram shows the strong right skew: the tallest bar is at 0 likes, and the bars drop off quickly as likes increase. Most posts receive very few likes: 862 of 2,400 posts (35.9%) have none, and 1,889 (78.7%) have 4 or fewer. A small number of posts receive many more likes than the rest, which also confirms that mean likes would be misleading.

```
# Group likes into three engagement levels:
# 0 = no engagement, 1-4 = some engagement, 5+ = high engagement
posts <- posts |>
  mutate(likes_group = cut(like_count,
                           breaks = c(-1, 0, 4, Inf),
                           labels = c("0 likes", "1-4 likes", "5+ likes")))

```

The cut-offs represent no engagement (0 likes), some engagement (1–4 likes) and high engagement (5+ likes).

```
# Contingency table: number of posts in each framing x engagement level
tab <- table(posts$framing, posts$likes_group)
tab
```

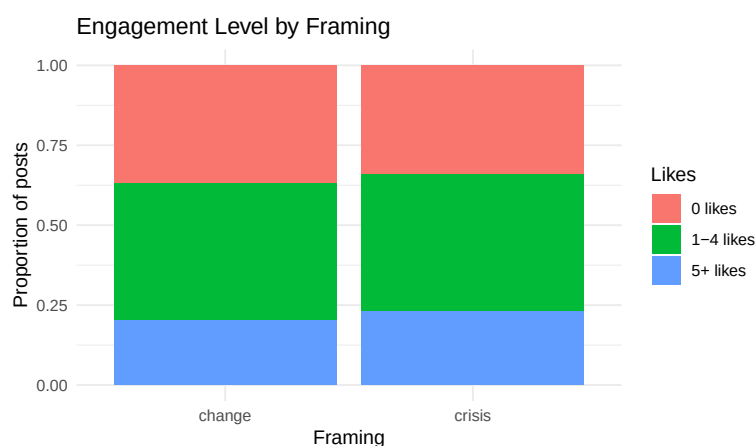
```
##
##           0 likes 1-4 likes 5+ likes
## change      559      648      306
## crisis      303      379      205
```

```
# Row proportions (fairer comparison, since there are more "change" posts than "crisis" posts)
round(prop.table(tab, 1), 3)
```

```
##
##           0 likes 1-4 likes 5+ likes
## change    0.369    0.428    0.202
## crisis    0.342    0.427    0.231
```

The proportions are very similar across the two framings. “Crisis” posts have a slightly higher share of highly liked posts (23.1% vs 20.2% with 5+ likes) and a slightly lower share of posts with no likes (34.2% vs 36.9%).

```
# Stacked bar chart: proportion of posts in each engagement level, by framing
ggplot(posts, aes(x = framing, fill = likes_group)) +
  geom_bar(position = "fill") +
  labs(title = "Engagement Level by Framing",
       x = "Framing", y = "Proportion of posts", fill = "Likes") +
  theme_minimal()
```



The stacked bar chart shows the proportion of posts in each engagement level for each framing. The two bars look very similar. The 5+ likes section (blue, bottom) is slightly taller for “crisis” posts (23.1% vs 20.2% “change”), which pushes the 1–4 likes section (green) slightly higher, although both green sections are about the same height (42.7% vs 42.9%). The 0 likes section (red, top) is correspondingly slightly smaller for “crisis” (34.2% vs 36.9%). This suggests framing has little effect on engagement, which the chi-squared test below checks formally.

```
# Run the chi-squared test of independence
test <- chisq.test(tab)

# Check the expected counts (should be at least 5 for the test to be valid)
round(test$expected, 1)
```

```
##
##           0 likes 1-4 likes 5+ likes
## change    543.4    647.4    322.1
## crisis    318.6    379.6    188.9
```

All expected counts are well above 5 (the smallest is 188.9), so the chi-squared approximation is valid.

```
# Show the chi-squared test result
test
```

```
##
## Pearson's Chi-squared test
##
## data:  tab
## X-squared = 3.3988, df = 2, p-value = 0.1828
```

```
# Simulated p-value (randomisation approach from Module 3)
# set.seed makes the simulated result the same every time
set.seed(3020)
chisq.test(tab, simulate.p.value = TRUE, B = 10000)
```

```
##
## Pearson's Chi-squared test with simulated p-value (based on 10000
## replicates)
##
## data:  tab
## X-squared = 3.3988, df = NA, p-value = 0.1819
```

Result and interpretation: The chi-squared test gives $\chi^2 = 3.40$ ($df = 2$), $p = 0.183$, and the simulated p-value (0.182) agrees. We therefore cannot reject H_0 : there is no evidence that engagement depends on framing. Although “crisis” posts were slightly more likely to receive 5+ likes, the difference is small and could be due to chance.

This suggests the framing label alone does not explain how much engagement a post receives. If framing alone doesn’t matter, perhaps the content of the posts does, which we explore through text analysis (2.3) and clustering (2.4).

Limitations: Likes depend on more than framing. Accounts with more followers naturally get more likes, and we did not control for this. The engagement cut-offs (0, 1–4, 5+) are also a judgement call; different cut-offs could give slightly different results.

2.3 Text/Content Analysis and Visualisation

Research question: Do “climate crisis” and “climate change” posts differ in the words and topics they use?

What? We explore the language used in the 2,400 clean posts: which words appear most often overall, and which words are most common in “climate crisis” posts compared with “climate change” posts.

Why? The hypothesis test (2.2) found no evidence that engagement depends on framing. Text analysis lets us check whether the two framings differ in **what** people talk about, even if they get similar engagement.

Missing and unsuitable records: Empty posts, duplicates, very short posts and posts without either phrase were already removed in 2.1, leaving 2,400 posts. This section prepares the **text** inside those posts for word analysis; the original posts are not changed.

```
# Load the cleaned posts from 2.1
posts <- readRDS("posts_clean.rds")

# Check for missing text
sum(is.na(posts$text))
```

```
## [1] 0
```

There is no missing text (0 posts), so all 2,400 posts can be analysed.

Text summaries

```
# Post length (number of words) by framing
posts |>
  group_by(framing) |>
  summarise(posts = n(),
            mean_words = round(mean(n_words), 1),
            median_words = median(n_words),
            max_words = max(n_words))
```

```
## # A tibble: 2 x 5
##   framing posts mean_words median_words max_words
##   <chr>   <int>      <dbl>         <int>      <int>
## 1 change  1513        33.1           35         61
## 2 crisis   887        34.1           36         64
```

Posts in both framings are a similar length. “Crisis” posts average 34.1 words (median 36) and “change” posts 33.1 words (median 35). The maximum of 61–64 words reflects Bluesky’s short post format. Any difference between the framings is therefore about *what* is said, not how much.

Cleaning the text

Before counting words, we remove things that would show up as meaningless “words”:

- **Links and @usernames**, otherwise punctuation removal turns them into junk like “http-swwwtheguardiancom”
- **Accented letters**: “ñ” is converted to “n” first, so “El Niño” (a climate pattern that appears often) is kept as “nino” instead of being cut to “nio”
- **Emojis and special characters**
- **Numbers and punctuation**
- **Stopwords** (common words such as “the”, “and”, “is”)
- **The search words** “climate”, “crisis”, “change” and their hashtag versions (#ClimateChange, #ClimateCrisis): every post contains them by design, so they would dominate without telling us anything
- **Generic filler words** such as “will”, “like”, “just”, “can” and “get”, which our first run showed dominated the top words without carrying meaning
- **“amp”** (left over from the “&” symbol) and **“bigearthdataai”** (an account that appears to post repeated automated content)

We then **stem** the words (e.g. “warming” and “warmed” become “warm”) so different forms of the same word are counted together. Stemmed words therefore appear in shortened form in the outputs, e.g. “peopl” (people), “energi” (energy), “secur” (security) and “futur” (future).

```
# Remove links and @usernames, convert ñ to n (so "El Niño" is kept as "nino"),
# then remove emojis/special characters
clean_text <- posts$text |>
  str_remove_all("http\\S+|www\\.\\.\\S+") |>
  str_remove_all("@\\S+") |>
  str_replace_all("[ñÑ]", "n") |>
  iconv(to = "ASCII", sub = "")
```

```
# Words to remove: stopwords, the search words (and hashtag versions),
# generic filler words that carry no meaning, and an automated account name
custom_stop <- c(stopwords("english"),
```

```
"climate", "crisis", "change", "climatechange", "climatecrisis", "amp",
"will", "like", "just", "can", "get", "one", "dont", "now", "also",
"even", "much", "way", "see", "know", "thing", "things", "want",
"let", "use", "still", "really", "that", "isnt", "doesnt", "look",
"come", "many", "lot", "well", "got", "going", "new", "make",
"say", "says", "said", "bigearthdataai")
```

```
# Build the corpus (one document per post) and clean it
corpus <- VCorpus(VectorSource(clean_text))
corpus <- tm_map(corpus, content_transformer(tolower)) # lowercase
corpus <- tm_map(corpus, removeNumbers) # remove numbers
corpus <- tm_map(corpus, removePunctuation) # remove punctuation
corpus <- tm_map(corpus, removeWords, custom_stop) # remove stopwords and custom words
corpus <- tm_map(corpus, stemDocument) # stem words
corpus <- tm_map(corpus, stripWhitespace) # remove extra spaces
```

Example: one post before cleaning

```
# Original text of the first post
posts$text[1]
```

```
## [1] "Sinn Fein are not a left wing party. Just populist bullshit. \n\nGet rid of USC? A
↪ progressive tax \nGet rid of property tax? An actual wealth tax \nPermanently reduce fuel prices ?
↪ In a climate crisis. \n\nA joke budget promising goodies to all to be paid for using Humm, Klarna
↪ and the credit card."
```

The same post after cleaning

```
# Cleaned text of the first post
as.character(corpus[[1]])
```

```
## [1] "sinn fein left wing parti populist bullshit rid usc progress tax rid properti tax actual
↪ wealth tax perman reduc fuel price joke budget promis goodi paid use humm klarna credit card"
```

The example shows the effect of cleaning: punctuation, stopwords (“are”, “not”, “a”), filler words (“just”, “get”) and the phrase “climate crisis” are removed, and words are stemmed (e.g. “party” → “parti”, “permanently” → “perman”, “promising” → “promis”). One limitation is visible: filler words are removed before stemming, so inflected forms such as “using” still appear after stemming as “use”.

Representing the text

We represent the text as a **term-document matrix**: each row is a word, each column is a post, and each cell counts how many times that word appears in that post (vector space model, Module 4). In 2.4 this matrix is weighted with TF-IDF for clustering.

```
# Term-document matrix
tdm <- TermDocumentMatrix(corpus)
tdm
```

```
## <<TermDocumentMatrix (terms: 6878, documents: 2400)>>
## Non-/sparse entries: 35974/16471226
## Sparsity : 100%
## Maximal term length: 93
## Weighting : term frequency (tf)
```

The term-document matrix contains 6,878 unique terms across 2,400 posts. Only 35,974 of its 16,507,200 cells are non-zero, so the matrix is over 99.7% sparse, and each post contains about 15 distinct terms on average. This is typical of short social media posts: most words appear in only a few posts, which is why TF-IDF weighting is used for clustering in 2.4.

```
# Convert to a matrix and count how often each word appears overall
m <- as.matrix(tdm)
word_freq <- sort(rowSums(m), decreasing = TRUE)
freq_df <- data.frame(word = names(word_freq), freq = word_freq)

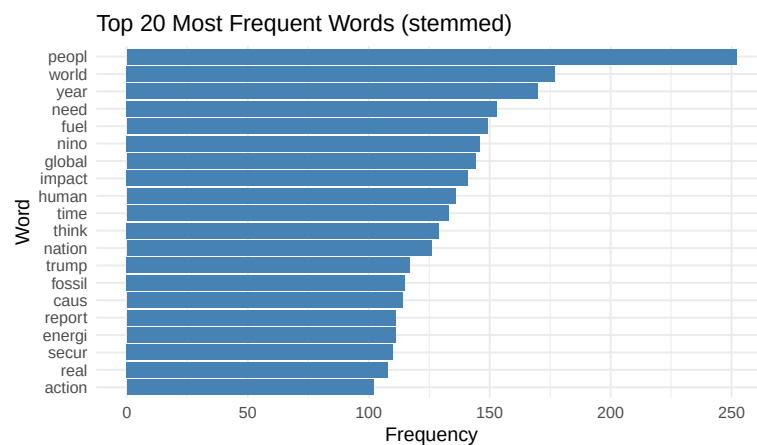
# Check for posts with no words left after cleaning
sum(colSums(m) == 0)
```

```
## [1] 2
```

Two posts contained no words after cleaning (likely because their text consisted only of stopwords, links or mentions). They are kept in the data but have no effect on word frequencies.

Visualisation 1: Most frequent words

```
# Bar chart of the 20 most frequent words
ggplot(head(freq_df, 20), aes(x = reorder(word, freq), y = freq)) +
  geom_col(fill = "steelblue") +
  coord_flip() +
  labs(title = "Top 20 Most Frequent Words (stemmed)",
       x = "Word", y = "Frequency") +
  theme_minimal()
```



After removing filler words, the most frequent terms point to several themes: **human impact** (“peopl”, “human”, “impact”, “caus”), **energy** (“fuel”, “fossil”, “energi”), **politics and security** (“trump”, “nation”, “secur”, “report”), **urgency** (“need”, “time”, “action”) and **extreme weather** (“nino”, for El Niño, which appears 146 times). The conversation is therefore not only about climate science, but also about energy, politics and people’s lives.

Visualisation 2: Word cloud of all posts

```
# Word cloud of the 150 most frequent words (bigger = more frequent)
set.seed(3020)
wordcloud(words = freq_df$word, freq = freq_df$freq,
          max.words = 150, random.order = FALSE,
          colors = brewer.pal(8, "Dark2"))
```


Climate change posts

```
# Word cloud of "change" posts
set.seed(3020)
wordcloud(words = names(change_freq), freq = change_freq,
          max.words = 100, random.order = FALSE,
          colors = brewer.pal(8, "Dark2"))
```



The clouds confirm the table. “peopl”, “world”, “year” and “fuel” are prominent in both. The “crisis” cloud gives much more space to “nation”, “report”, “secur” and “nino”, along with urgent words such as “death”, “war” and “warn”, while “trump” does not appear in it. In the “change” cloud, after “peopl” and “year”, the next largest words include “think”, “need”, “human” and “trump”, and “hoax” and “real” also stand out; “hoax” does not appear in the “crisis” cloud. Political words such as “vote”, “polici” and “govern” also appear only in the “change” cloud. This suggests the “change” framing is more often used in debates about politics and whether climate change is real, while “crisis” is more often used alongside news reports, national security and extreme weather.

2.4 Clustering

Research question: Can posts be grouped into distinct themes based on their words, and are the two framings distributed differently across these themes?

What? We investigate whether the 2,400 posts form meaningful groups (clusters) based on the words they use, and what theme each cluster represents.

Why? The text analysis (2.3) suggested several overlapping themes (energy, politics and security, human impact, extreme weather, debate over whether climate change is real). Clustering identifies these themes directly from the posts, without using the framing labels. We can then check whether “crisis” and “change” posts fall into different themes.

```
# Load the cleaned posts from 2.1
posts <- readRDS("posts_clean.rds")
```

Representation: cleaning the text (same steps as 2.3)

We use exactly the same text cleaning as 2.3, so both sections analyse the same words.

```

# Remove links and @usernames, convert ñ to n, then remove emojis/special characters
clean_text <- posts$text |>
  str_remove_all("http\\S+|www\\.\\S+") |>
  str_remove_all("@\\S+") |>
  str_replace_all("[ñÑ]", "n") |>
  iconv(to = "ASCII", sub = "")

# Same custom stopwords list as 2.3
custom_stop <- c(stopwords("english"),
  "climate", "crisis", "change", "climatechange", "climatecrisis", "amp",
  "will", "like", "just", "can", "get", "one", "dont", "now", "also",
  "even", "much", "way", "see", "know", "thing", "things", "want",
  "let", "use", "still", "really", "that", "isnt", "doesnt", "look",
  "come", "many", "lot", "well", "got", "going", "new", "make",
  "say", "says", "said", "bigearthdataai")

# Build and clean the corpus (one document per post)
corpus <- VCorpus(VectorSource(clean_text))
corpus <- tm_map(corpus, content_transformer(tolower))
corpus <- tm_map(corpus, removeNumbers)
corpus <- tm_map(corpus, removePunctuation)
corpus <- tm_map(corpus, removeWords, custom_stop)
corpus <- tm_map(corpus, stemDocument)
corpus <- tm_map(corpus, stripWhitespace)

```

Representation: document-term matrix and TF-IDF

```

# Document-term matrix: one row per post, one column per word
dtm <- DocumentTermMatrix(corpus)
dtm

```

```

## <<DocumentTermMatrix (documents: 2400, terms: 6878)>>
## Non-/sparse entries: 35974/16471226
## Sparsity           : 100%
## Maximal term length: 93
## Weighting          : term frequency (tf)

```

The document-term matrix has 2,400 posts and 6,878 terms, the same vocabulary as in 2.3.

Most of these terms appear in only one or two posts. Rare words cannot help group posts together, and they add noise to the distances, so we keep only terms that appear in at least 0.5% of posts (about 12 or more posts).

```

# Keep only terms that appear in at least 0.5% of posts
dtm_small <- removeSparseTerms(dtm, 0.995)
dtm_small

```

```

## <<DocumentTermMatrix (documents: 2400, terms: 667)>>
## Non-/sparse entries: 21331/1579469
## Sparsity           : 99%
## Maximal term length: 13
## Weighting          : term frequency (tf)

```

This reduces the vocabulary from 6,878 to 667 terms. The matrix is still very sparse: only 21,331 of its 1,600,800 cells (1.3%) are non-zero.

```
# Posts with none of the remaining terms cannot be compared, so they are removed
tf <- as.matrix(dtm_small)
keep <- rowSums(tf) > 0
sum(!keep)
```

```
## [1] 15
```

15 posts contained none of the 667 remaining terms, so they are removed, leaving 2,385 posts for clustering.

We weight the matrix with **TF-IDF** (Module 4). It gives more weight to words that are frequent in a post but rare across all posts, so distinctive words count more than words that appear everywhere. `tm`'s TF-IDF also adjusts for post length, so longer posts don't dominate.

```
# TF-IDF weighting, then keep only the posts with at least one remaining term
X <- as.matrix(weightTfIdf(dtm_small))[keep, TRUE]
posts_k <- posts[keep, ]
dim(X)
```

```
## [1] 2385 667
```

Distance measure: cosine distance

We use **cosine distance** (Modules 4 and 5). It compares posts by the *direction* of their word vectors, i.e. which words they use, regardless of length. Euclidean distance would treat two posts as different simply because one is longer. Cosine distance is 0 for posts using the same words in the same proportions, and 1 for posts with no words in common.

```
# Cosine distance = 1 - cosine similarity
X_norm <- X / sqrt(rowSums(X^2)) # scale each post's vector to length 1
cos_dist <- as.dist(1 - X_norm %*% t(X_norm))
```

First attempt: hierarchical clustering

We first tried **hierarchical clustering** (Module 6), because it works directly on the cosine distance matrix. We compared two linkage methods:

- **Complete linkage:** the distance between clusters is the *largest* distance between their posts
- **Group average linkage:** the distance between clusters is the *average* distance between their posts

```
# Hierarchical clustering with two linkage methods
hc_complete <- hclust(cos_dist, method = "complete")
hc_average <- hclust(cos_dist, method = "average")
```

For each linkage, we cut the tree into 2 to 8 clusters and compared the cluster sizes. Useful clusters should be reasonably balanced: a solution with one huge cluster and several tiny ones does not summarise the data well.

```
# Cluster sizes (largest to smallest) for k = 2 to 8, for each linkage method
sizes <- function(hc, k) paste(sort(table(cutree(hc, k))), decreasing = TRUE), collapse = " / ")

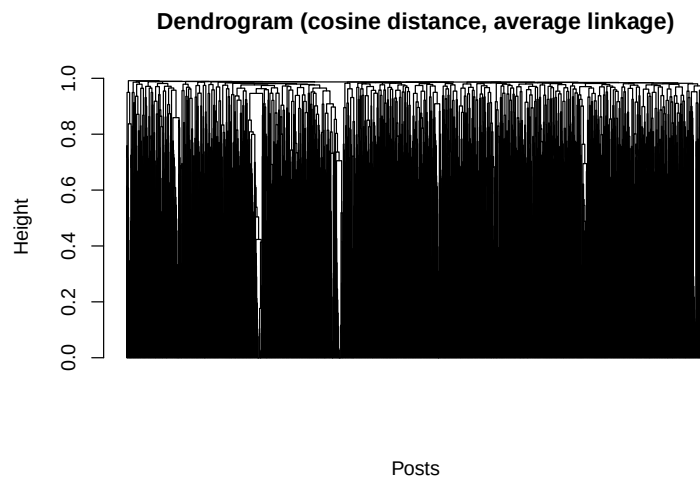
size_table <- data.frame(
  k = 2:8,
  complete = sapply(2:8, function(k) sizes(hc_complete, k)),
  average = sapply(2:8, function(k) sizes(hc_average, k))
)
kable(size_table, caption = "Hierarchical clustering: cluster sizes for each k and linkage method")
```

Table 6: Hierarchical clustering: cluster sizes for each k and linkage method

k	complete	average
2	2383 / 2	2364 / 21
3	2380 / 3 / 2	2310 / 54 / 21
4	2375 / 5 / 3 / 2	2158 / 152 / 54 / 21
5	2372 / 5 / 3 / 3 / 2	2105 / 152 / 54 / 53 / 21
6	2370 / 5 / 3 / 3 / 2 / 2	2051 / 152 / 54 / 54 / 53 / 21
7	2366 / 5 / 4 / 3 / 3 / 2 / 2	2019 / 152 / 54 / 54 / 53 / 32 / 21
8	2363 / 5 / 4 / 3 / 3 / 3 / 2 / 2	2019 / 116 / 54 / 54 / 53 / 36 / 32 / 21

Neither linkage produces useful clusters. **Complete linkage** keeps at least 2,363 of the 2,385 posts in one cluster for every k, and only splits off tiny groups of 2–5 posts. **Average linkage** is better, but one cluster still holds at least 2,019 posts (85%) at every k.

```
# Dendrogram of the average linkage tree
plot(hc_average, labels = FALSE, hang = -1,
     main = "Dendrogram (cosine distance, average linkage)",
     xlab = "Posts", sub = "")
```



The dendrogram confirms this. Almost all merges happen at heights between about 0.8 and 1.0, meaning most posts are joined to the tree only when clusters are already very dissimilar. Only a few narrow branches reach lower heights; these are small groups of near-identical posts, which form the tiny clusters in the table above.

The reason is the data itself. After removing rare terms, each post contains only about 9 distinct terms out of 667, so many pairs of posts share no words at all and have a cosine distance close to 1.

With so many distances close to 1, hierarchical clustering has little structure to work with: it merges a few near-identical posts early and leaves almost everything else in one large group.

Final method: k-means on normalised TF-IDF vectors

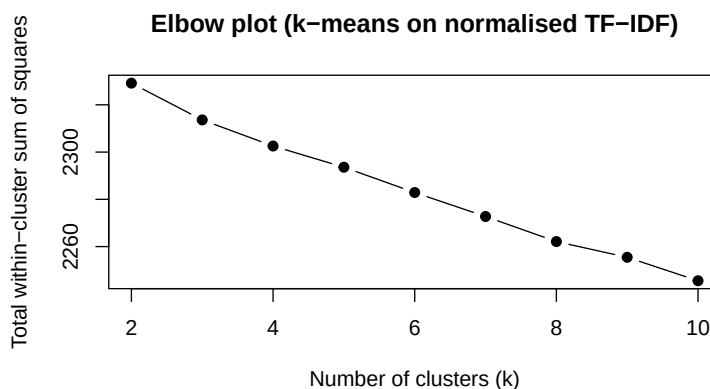
We therefore use **k-means** (Module 6) on the TF-IDF vectors after scaling each post's vector to length 1 (**X_norm**). For vectors of length 1, the squared Euclidean distance used by k-means equals $2 \times$ the cosine distance, so k-means on **X_norm** still groups posts by cosine similarity, the same distance measure as above. Unlike hierarchical clustering, k-means builds clusters around central points (centroids), which tends to give more balanced groups. We use 20 random starts (**nstart = 20**) to avoid a poor solution.

Choosing the number of clusters

We run k-means for $k = 2$ to 10 and record the **total within-cluster sum of squares** (how spread out the posts are within their clusters). This always decreases as k increases. We look for an **elbow**, where adding more clusters stops giving a large improvement. We also check the cluster sizes for balance.

```
# Run k-means for k = 2 to 10 (same random seed for each, so results are reproducible)
fits <- lapply(2:10, function(k) {
  set.seed(3020)
  kmeans(X_norm, centers = k, nstart = 20, iter.max = 50)
})
```

```
# Elbow plot: total within-cluster sum of squares for each k
wss <- sapply(fits, function(f) f$tot.withinss)
plot(2:10, wss, type = "b", pch = 19,
     main = "Elbow plot (k-means on normalised TF-IDF)",
     xlab = "Number of clusters (k)", ylab = "Total within-cluster sum of squares")
```



The total within-cluster sum of squares decreases steadily and almost linearly from $k = 2$ to $k = 10$, with no clear elbow, and the overall reduction is small relative to the total. This shows that the posts do not divide into strongly separated groups, which is consistent with how sparse short posts are. Since the elbow method does not identify a single best k , we choose k using cluster balance and how interpretable the clusters are.

```
# Cluster sizes (largest to smallest) for each k
kable(data.frame(k = 2:10,
                 sizes = apply(fits, function(f) paste(sort(f$size, decreasing = TRUE), collapse = "
                 ↪ / "))),
       caption = "k-means: cluster sizes for each k")
```

Table 7: k-means: cluster sizes for each k

k	sizes
2	2329 / 56
3	2247 / 82 / 56
4	2109 / 151 / 69 / 56
5	1974 / 197 / 110 / 56 / 48
6	1960 / 127 / 105 / 79 / 63 / 51
7	1956 / 129 / 105 / 79 / 53 / 51 / 12
8	1509 / 280 / 226 / 128 / 84 / 60 / 50 / 48
9	1374 / 320 / 205 / 138 / 121 / 96 / 50 / 48 / 33
10	1341 / 200 / 196 / 180 / 140 / 122 / 82 / 60 / 34 / 30

We chose $k = 5$. Unlike hierarchical clustering, k-means separates several medium-sized groups from the main body of posts: at $k = 5$ the clusters contain 1,974, 197, 110, 56 and 48 posts. Each of the four smaller clusters has a distinct and interpretable theme (see below). Larger values of k give more evenly sized clusters, but with no elbow there is no statistical reason to prefer them, and more clusters are harder to interpret and present. The choice of k is therefore a judgement, which we note as a limitation.

```
# Chosen number of clusters (decided from the elbow plot and size table above)
k <- 5
clusters <- fits[[k - 1]]$cluster
table(clusters)
```

```
## clusters
##      1      2      3      4      5
## 197    56   110    48 1974
```

Cluster themes

To identify each cluster's theme, we find the words with the highest average weight within the cluster: the words most characteristic of its posts.

```
# Top 8 most characteristic words in each cluster (highest average normalised TF-IDF)
top_terms <- sapply(1:k, function(j) {
  avg <- colMeans(X_norm[clusters == j, TRUE, drop = FALSE])
  paste(names(sort(avg, decreasing = TRUE))[1:8], collapse = ", ")
})

kable(data.frame(cluster = 1:k,
                 posts = as.vector(table(clusters)),
                 top_words = top_terms),
       caption = "Size and most characteristic words of each cluster")
```

Table 8: Size and most characteristic words of each cluster

cluster	posts	top_words
1	197	fuel, hoax, fossil, industri, compani, justic, alito, case
2	56	secur, nation, suppress, starmer, share, find, damn, report
3	110	think, peopl, time, solv, work, someth, start, year
4	48	death, nino, project, toll, top, nigeria, month, temperatur
5	1974	peopl, world, need, real, human, global, year, impact

Based on their most characteristic words, the five clusters can be described as:

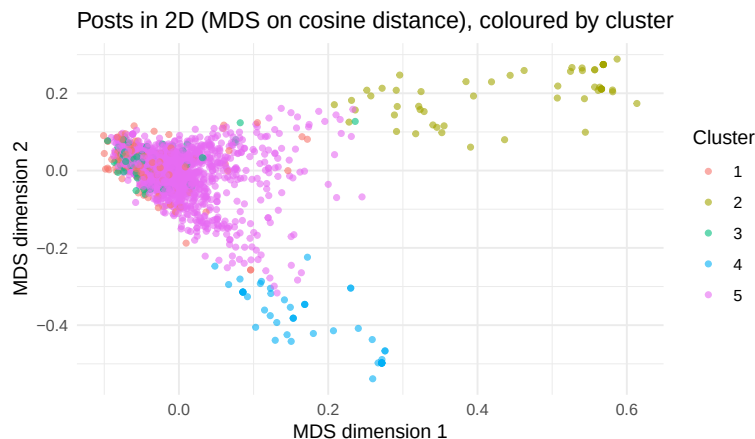
- Cluster 1 (197 posts), fossil fuels, industry and legal cases: “fuel”, “fossil”, “industri”, “compani”, together with “justic”, “alito” and “case”, which suggest discussion of a court case involving fossil fuel companies, and “hoax”, which suggests scepticism.
- Cluster 2 (56 posts), national security report: “secur”, “nation”, “suppress”, “starmer” and “report” suggest a UK news story about a suppressed national security report, involving Prime Minister Keir Starmer.
- Cluster 3 (110 posts), opinions and solutions: “think”, “peopl”, “solv”, “work”, “start” suggest personal views on what should be done.
- Cluster 4 (48 posts), extreme weather and deaths: “death”, “nino”, “toll”, “project”, “temperatur” suggest reports of projected death tolls linked to El Niño and rising temperatures.
- Cluster 5 (1,974 posts, 82.8%), general climate discussion: broad words such as “peopl”, “world”, “need”, “real”, “global” and “impact”, the same top words as the whole dataset in 2.3.

Visualising the clusters (MDS)

We visualise the clusters with **multidimensional scaling** (Module 5). MDS places the posts in two dimensions so that the distances between points approximate their cosine distances. Unlike PCA, it works with any distance measure.

```
# Classical MDS on the cosine distances, reduced to 2 dimensions
mds <- cmdscale(cos_dist, k = 2)
mds_df <- data.frame(dim1 = mds[, 1], dim2 = mds[, 2],
                     cluster = factor(clusters),
                     framing = posts_k$framing)

ggplot(mds_df, aes(x = dim1, y = dim2, colour = cluster)) +
  geom_point(alpha = 0.6, size = 1.2) +
  labs(title = "Posts in 2D (MDS on cosine distance), coloured by cluster",
       x = "MDS dimension 1", y = "MDS dimension 2", colour = "Cluster") +
  theme_minimal()
```



The MDS plot shows a dense core containing most posts, where clusters 5 (general discussion), 1 (fossil fuels and legal cases) and 3 (opinions and solutions) overlap. Two clusters are clearly separated. Cluster 2 (national security report) forms a distinct arm towards the upper right, and cluster 4 (extreme weather and deaths) forms a distinct arm towards the bottom. These two clusters use vocabulary that is very different from the rest of the posts. The overlap in the core is partly because MDS shows only two dimensions of the cosine distances, so clusters that overlap here may still be separated in the full space.

Linking clusters to framing

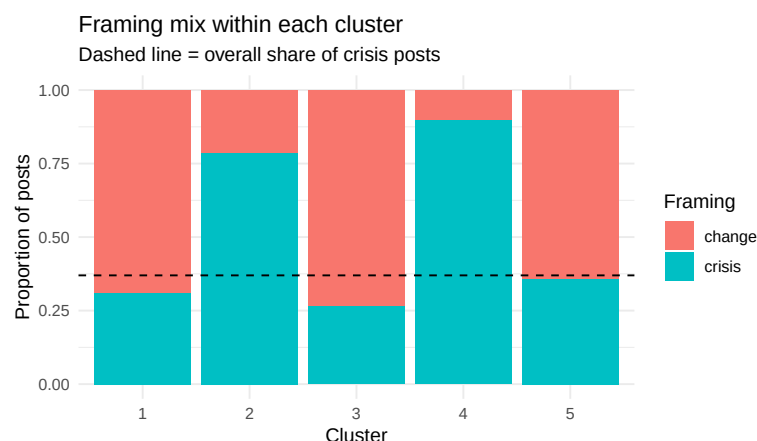
```
# Number of posts of each framing in each cluster
tab_cf <- table(cluster = clusters, framing = posts_k$framing)
tab_cf
```

```
##      framing
## cluster change crisis
##      1      136      61
##      2       12      44
##      3       81      29
##      4        5      43
##      5     1269     705
```

```
# Proportion of "change" and "crisis" posts within each cluster
round(prop.table(tab_cf, 1), 3)
```

```
##      framing
## cluster change crisis
##      1  0.690  0.310
##      2  0.214  0.786
##      3  0.736  0.264
##      4  0.104  0.896
##      5  0.643  0.357
```

```
# Stacked bar chart: framing mix within each cluster
ggplot(mds_df, aes(x = cluster, fill = framing)) +
  geom_bar(position = "fill") +
  geom_hline(yintercept = mean(posts_k$framing == "crisis"), linetype = "dashed") +
  labs(title = "Framing mix within each cluster",
       subtitle = "Dashed line = overall share of crisis posts",
       x = "Cluster", y = "Proportion of posts", fill = "Framing") +
  theme_minimal()
```



Overall, 37.0% of the clustered posts use “crisis” framing (dashed line). Clusters 2 and 4 are dominated by “crisis” posts: 78.6% of the national security cluster and 89.6% of the extreme weather cluster. In contrast, “crisis” is under-represented in clusters 1 (31.0%) and 3 (26.4%), the fossil fuel and opinion clusters. The general discussion cluster (35.7%) is close to the overall share.

To check whether the framing mix really differs between clusters, we use the chi-squared test of independence (Module 3), as in 2.2.

```
# Chi-squared test: are cluster and framing independent?
test_cf <- chisq.test(tab_cf)
round(test_cf$expected, 1) # expected counts should be at least 5
```

```
##      framing
## cluster change crisis
##    1  124.1  72.9
##    2   35.3  20.7
##    3   69.3  40.7
##    4   30.2  17.8
##    5 1244.0 730.0
```

```
test_cf
```

```
##
## Pearson's Chi-squared test
##
## data:  tab_cf
## X-squared = 108.29, df = 4, p-value < 2.2e-16
```

H0: cluster and framing are independent. H1: cluster and framing are not independent. All expected counts are above 5 (the smallest is 17.8), so the test is valid. The chi-squared test gives $\chi^2 = 108.29$ (df = 4), $p < 0.001$, so we reject H0: the framing mix differs significantly between clusters. “Crisis” framing is concentrated in posts about specific news events (the security report and El Niño deaths), while “change” framing is more common in opinion and debate posts.

Linking clusters to engagement

```
# Engagement in each cluster
posts_k |>
  mutate(cluster = factor(clusters)) |>
  group_by(cluster) |>
  summarise(posts = n(),
            median_likes = median(like_count),
            pct_5plus_likes = round(mean(like_count >= 5) * 100, 1))
```

```
## # A tibble: 5 x 4
##   cluster posts median_likes pct_5plus_likes
##   <fct>   <int>         <dbl>         <dbl>
## 1 1         197             1           19.8
## 2 2          56             1           19.6
## 3 3         110             1           27.3
## 4 4          48             1           27.1
## 5 5        1974             1           21.1
```

Every cluster has a median of 1 like. The share of posts with 5 or more likes ranges from 19.6% (cluster 2) to 27.3% (cluster 3), so engagement is broadly similar across themes. Clusters 3 and 4 are

slightly higher, but they are small (110 and 48 posts), so these differences may not be meaningful. This is consistent with 2.2: neither framing nor theme is strongly linked to engagement.

Limitations: the clusters are only weakly separated (no clear elbow, one large general cluster), the choice of $k = 5$ is a judgement, and the two smallest clusters may each reflect a single news story rather than a lasting theme.

2.5 Network Analysis

Research question: Which accounts occupy the most important positions in the Bluesky reply network around “climate change” and “climate crisis”, and do the two framings differ in how much reply interaction they involve?

What? We build a network of reply relationships between Bluesky accounts and identify the most central accounts.

Why? 2.2–2.4 analysed what posts say and how many likes they get. The network shows who interacts with whom, and whether the conversation is one connected discussion or many separate ones.

Nodes and edges: A **node** is a Bluesky account. A **directed edge** from account A to account B means that A replied to a post written by B. Repeated replies from A to B are combined into one edge, with a **weight** equal to the number of replies.

Directed or undirected? The graph is **directed**. A reply has a clear sender and receiver: if A replies to B, B has not necessarily replied to A. This differs from, for example, Facebook friendships, which are mutual and therefore undirected.

Data used: We use the raw posts (`posts_raw.rds`) rather than the 2,400 cleaned posts, because the cleaning steps in 2.1 (such as removing short posts and posts without exactly one phrase) would remove many replies and make the network even smaller. Duplicate posts (found by both searches) are removed first, and each post’s framing is labelled from its text using the same rule as 2.1.

```
# Load raw posts and remove duplicates (posts returned by both searches)
raw <- readRDS("posts_raw.rds") |>
  distinct(uri, .keep_all = TRUE)

# Label each post's framing from its text (same rule as 2.1)
raw <- raw |>
  mutate(text_lower = str_to_lower(text),
         framing = case_when(
           str_detect(text_lower, "climate crisis") & str_detect(text_lower, "climate change") ~
             ↪ "both",
           str_detect(text_lower, "climate crisis") ~ "crisis",
           str_detect(text_lower, "climate change") ~ "change",
           TRUE ~ "neither"))

# Number of posts after removing duplicates
nrow(raw)
```

```
## [1] 4129
```

Removing duplicate posts leaves 4,129 posts, 19 fewer than the 4,148 collected.

Constructing the reply network

```

# ID (DID) of the author of each collected post
raw$source_did <- sapply(raw$author_data, function(x) {
  if (is.null(x$did)) NA_character_ else x$did
})

# ID of the account being replied to, taken from the parent post's address
raw$target_did <- str_extract(raw$in_reply_to, "did:plc:[^/]+")

# Keep valid replies only (both IDs known, and remove self-replies)
reply_events <- raw |>
  filter(!is.na(source_did), !is.na(target_did), source_did != target_did) |>
  transmute(source_did, target_did, framing)

# Combine repeated replies between the same two accounts into one weighted edge
reply_edges <- reply_events |>
  count(source_did, target_did, name = "weight")

# Build the directed, weighted graph
g <- graph_from_data_frame(reply_edges, directed = TRUE)
g

```

```

## IGRAPH 7c9c032 DNW- 1249 779 --
## + attr: name (v/c), weight (e/n)
## + edges from 7c9c032 (vertex names):
## [1] did:plc:23ip7qinl7um5a6u7x4wgfna->did:plc:bak7f4b3jsiqlpyo6o4ejaji
## [2] did:plc:2b63hv6xpuuwyby5e2smvezwk->did:plc:x6npzh35qsuuxshx7vhbtpn
## [3] did:plc:2cd32akwobpz3nsx55vk6q3z->did:plc:7qqkq2zdwq4j5jingukgtuky
## [4] did:plc:2d3la6gstrt7tfq3psughz4x->did:plc:lia4ywzl2c2kt4dn3kzbywog
## [5] did:plc:2dbmwvxxvittrfxhgw2fht643->did:plc:zftsnmv3oeatoag66zia5sfu
## [6] did:plc:2dl12i7bqhuwime2aznm7ulab->did:plc:sngu3sdsec2czdysb4xro3u5
## [7] did:plc:2et64cn4oidgnrdow26vfets->did:plc:swgxwbir4rrimj57pf75jckf
## [8] did:plc:2h23ypiewjln3dvpznt6o5xu->did:plc:gfu4zyznquceeb5xznbi5u
## + ... omitted several edges

```

The reply network contains **1249 accounts (nodes)** and **779 directed edges**, built from **815 reply events**.

```

# Readable labels: use the handle where the account is a collected author, otherwise its ID
did_lookup <- raw |>
  filter(!is.na(source_did), !is.na(author_handle)) |>
  distinct(source_did, .keep_all = TRUE) |>
  select(did = source_did, handle = author_handle)

vertex_df <- tibble(did = V(g)$name) |>
  left_join(did_lookup, by = "did") |>
  mutate(label = ifelse(is.na(handle), did, handle))
V(g)$label <- vertex_df$label

# Each account's main framing (the framing it uses most often in its own posts)
author_framing <- raw |>
  filter(framing %in% c("crisis", "change"), !is.na(source_did)) |>
  count(source_did, framing) |>
  group_by(source_did) |>
  slice_max(n, n = 1, with_ties = FALSE) |>
  ungroup()

V(g)$framing <- author_framing$framing[match(V(g)$name, author_framing$source_did)]

# Node colours: red = mostly crisis, blue = mostly change,
# grey = neither phrase or not a collected author
node_cols <- ifelse(is.na(V(g)$framing), "grey85",
  ifelse(V(g)$framing == "crisis", "firebrick", "steelblue"))

```

Network properties

```
# Connected components (weak = ignoring direction, strong = following direction)
weak_comp  <- components(g, mode = "weak")
strong_comp <- components(g, mode = "strong")

# Degrees: in = replies received from different accounts, out = accounts replied to
in_deg  <- degree(g, mode = "in")
out_deg <- degree(g, mode = "out")
total_deg <- degree(g, mode = "all")

# Summary table of network properties
network_summary <- tibble(
  Measure = c("Nodes", "Directed edges", "Reply events", "Density",
    "Weakly connected components", "Largest weak component (nodes)",
    "Strongly connected components",
    "Median in-degree", "Maximum in-degree",
    "Median out-degree", "Maximum out-degree"),
  Value = c(vcount(g), ecount(g), nrow(reply_events),
    format(round(edge_density(g, loops = FALSE), 5), scientific = FALSE),
    weak_comp$no, max(weak_comp$size), strong_comp$no,
    median(in_deg), max(in_deg), median(out_deg), max(out_deg)) |> as.character()
)
kable(network_summary, caption = "Properties of the reply network")
```

Table 9: Properties of the reply network

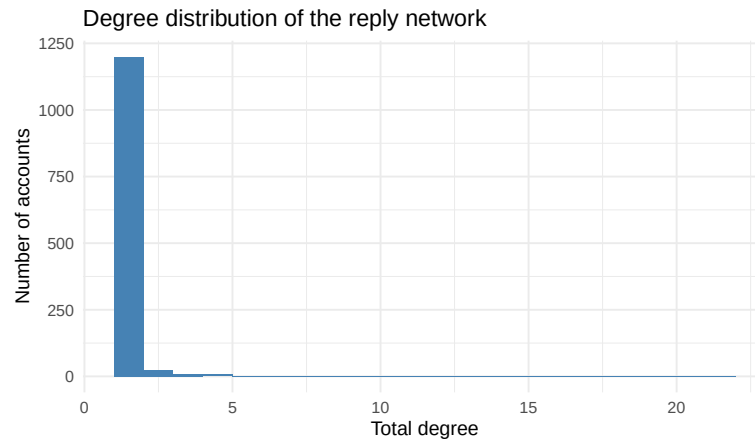
Measure	Value
Nodes	1249
Directed edges	779
Reply events	815
Density	0.0005
Weakly connected components	488
Largest weak component (nodes)	92
Strongly connected components	1232
Median in-degree	1
Maximum in-degree	18
Median out-degree	1
Maximum out-degree	22

The reply network contains 1,249 accounts and 779 directed edges, built from 815 reply events (36 replies were repeats between the same two accounts). The density is 0.0005, so only about 0.05% of all possible account-to-account reply links are observed. The network is highly fragmented: it splits into 488 weakly connected components, and the largest contains only 92 accounts (7.4% of all accounts). There are 1,232 strongly connected components among 1,249 accounts, which means almost no accounts are linked by two-way reply chains: replies mostly go one way and are not answered. The median account has an in-degree and out-degree of 1, while the maximum in-degree is 18 and the maximum out-degree is 22.

Degree distribution

```
# Histogram of total degree (number of distinct accounts each account is connected to)
ggplot(tibble(total_degree = as.numeric(total_deg)), aes(x = total_degree)) +
  geom_histogram(binwidth = 1, boundary = 0, fill = "steelblue") +
```

```
labs(title = "Degree distribution of the reply network",
     x = "Total degree", y = "Number of accounts") +
theme_minimal()
```



```
# Share of accounts (%) with low and high degree
tibble(degree_1 = round(mean(total_deg == 1) * 100, 1),
       degree_2_to_4 = round(mean(total_deg >= 2 & total_deg <= 4) * 100, 1),
       degree_5_plus = round(mean(total_deg >= 5) * 100, 1))
```

```
## # A tibble: 1 x 3
##   degree_1 degree_2_to_4 degree_5_plus
##   <dbl>      <dbl>      <dbl>
## 1      87.3        11.4         1.4
```

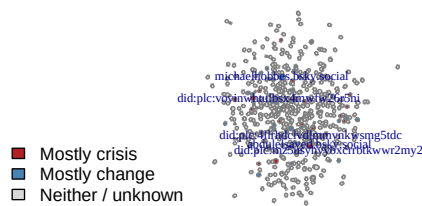
The degree distribution is extremely skewed. 87.3% of accounts are connected to only one other account, 11.4% to 2–4 accounts, and only 1.4% to 5 or more. This “many accounts with few connections, a few hubs” pattern is typical of social networks and matches the Barabási–Albert model from Module 7.

Full network

```
# Full network: size = in-degree, colour = main framing,
# labels for the 5 most replied-to accounts only (so they are readable)
set.seed(3020)
top5_in <- names(sort(in_deg, decreasing = TRUE))[1:5]
vertex_labels <- ifelse(V(g)$name %in% top5_in, V(g)$label, NA)

plot(g,
     layout = layout_with_fr(g),           # Fruchterman-Reingold layout
     vertex.size = 2 + 2 * log1p(in_deg),  # bigger = more replies received
     vertex.label = vertex_labels,
     vertex.label.cex = 0.55,
     vertex.color = node_cols,
     vertex.frame.color = "grey50",
     edge.width = 0.3 + log1p(E(g)$weight), # thicker = more replies between the pair
     edge.arrow.size = 0.15,
     edge.color = "grey65",
     main = "Full Bluesky climate reply network")
legend("bottomleft", legend = c("Mostly crisis", "Mostly change", "Neither / unknown"),
     fill = c("firebrick", "steelblue", "grey85"), bty = "n", cex = 0.8)
```

Full Bluesky climate reply network



The full network shows many small, separate groups rather than one connected conversation. Most accounts appear as isolated pairs or small stars (several accounts replying to one account). Most nodes are grey, because many replies do not themselves use either phrase, and many replied-to accounts are not collected authors.

Subgraph: largest connected component

The full network contains many small, disconnected groups. To examine the most connected part of the conversation, we take the **largest weakly connected component**: the biggest group of accounts linked by replies (ignoring direction). This is the largest continuous reply structure in the data, so it is the best place to see how accounts interact.

```
# Extract the largest weakly connected component
largest_id <- which.max(weak_comp$csizes)
g_lcc <- induced_subgraph(g, vids = V(g)[weak_comp$membership == largest_id])

# Number of nodes and edges in the largest component
c(nodes = vcount(g_lcc), edges = ecount(g_lcc))

## nodes edges
##      92    93

# Largest component: size = in-degree, colour = main framing,
# labels for the 10 most connected accounts
set.seed(3020)
lcc_in    <- degree(g_lcc, mode = "in")
lcc_total <- degree(g_lcc, mode = "all")
top10_lcc <- names(sort(lcc_total, decreasing = TRUE))[1:10]
lcc_labels <- ifelse(V(g_lcc)$name %in% top10_lcc, V(g_lcc)$label, NA)
lcc_cols <- ifelse(is.na(V(g_lcc)$framing), "grey85",
                  ifelse(V(g_lcc)$framing == "crisis", "firebrick", "steelblue"))

plot(g_lcc,
     layout = layout_with_fr(g_lcc),
     vertex.size = 4 + 3 * log1p(lcc_in),
     vertex.label = lcc_labels,
     vertex.label.cex = 0.7,
     vertex.color = lcc_cols,
     vertex.frame.color = "grey50",
     edge.width = 0.5 + log1p(E(g_lcc)$weight),
     edge.arrow.size = 0.2,
     edge.color = "grey60",
     main = "Largest connected reply community")
legend("bottomleft", legend = c("Mostly crisis", "Mostly change", "Neither / unknown"),
     fill = c("firebrick", "steelblue", "grey85"), bty = "n", cex = 0.8)
```

Largest connected reply community



The largest component has 92 accounts and 93 edges, almost a tree (a tree with 92 nodes has exactly 91 edges), so it contains very few closed loops of conversation. It is made up of several star-shaped groups, each centred on a heavily replied-to account, linked by a few connecting accounts. In the star around `abdulaysayed.bsky.social`, most replying accounts are red (mostly “crisis”), while several accounts in the group around the central `did:plc:4llrhdcivllmnykwsmg5tdc` account are blue (mostly “change”). This suggests that accounts replying to the same post often use the same framing.

Centrality

We use two centrality measures, plus PageRank as a supporting measure:

1. **In-degree** (Module 7): the number of different accounts that replied to an account. A high in-degree means the account attracts discussion.
2. **Betweenness** (Module 7): how often an account lies on the shortest paths between other accounts. A high value means the account bridges different parts of the network. We report it for the directed network and, because directed paths turn out to be extremely rare, also for the undirected network, which shows which accounts link groups together regardless of reply direction. Edge weights are not used, because in `igraph` weights are treated as distances.
3. **PageRank** (Module 8): the long-run probability of a random surfer landing on an account, following replies with probability $= 0.8$ (as in Module 8) and jumping randomly otherwise. It rewards accounts that receive replies from other important accounts.

```
# Centrality measures for every account
cent <- tibble(
  account      = V(g)$label,
  framing      = V(g)$framing,
  in_degree    = as.numeric(in_deg),
  out_degree   = as.numeric(out_deg),
  # undirected betweenness: bridging between groups, ignoring reply direction
  betweenness_undirected = round(betweenness(g, directed = FALSE, weights = NA,
                                             normalized = TRUE), 5),
  # PageRank with lambda = 0.8 (Module 8), using reply counts as edge weights
  pagerank     = round(page_rank(g, directed = TRUE, damping = 0.8,
                                weights = E(g)$weight)$vector, 4)
)
```

```
# Top 10 accounts by in-degree
cent |> arrange(desc(in_degree), desc(pagerank)) |> slice_head(n = 10) |>
  kable(format = "latex", booktabs = TRUE,
        caption = "Top 10 accounts by in-degree") |>
  kable_styling(latex_options = c("scale_down", "hold_position"))
```

Table 10: Top 10 accounts by in-degree

account	framing	in_degree	out_degree	betweenness_undirected	pagerank
did:plc:4llrhdclvdlmmynkwsmsg5tdc	NA	18	0	0.00351	0.0076
abdulelsayed.bsky.social	crisis	13	0	0.00130	0.0057
michaelhobbes.bsky.social	change	11	0	0.00007	0.0052
did:plc:vovinvhtulbsx4mwfw26r5ni	NA	6	0	0.00009	0.0025
did:plc:kx375dszrbvqtu7o4xxxmojn	NA	5	0	0.00004	0.0031
whstancil.bsky.social	change	5	0	0.00002	0.0028
did:plc:mz5usyhyvxoecrrbtkwwr2my2	NA	5	0	0.00001	0.0027
did:plc:uo2fna47c4v6zcnklxfhcvjb	NA	5	0	0.00045	0.0023
zackpolanski.bsky.social	crisis	4	0	0.00002	0.0027
davidshoebridge.bsky.social	crisis	4	0	0.00001	0.0022

The most replied-to account received replies from 18 different accounts, but its own posts were not in our sample, so only its ID is shown. The next most replied-to accounts appear to be public figures such as politicians and political commentators (e.g. abdulelsayed, michaelhobbes, whstancil, zackpolanski, davidshoebridge). All ten have an out-degree of 0: they attract replies but did not reply to anyone in this sample, which is consistent with a broadcast pattern where prominent accounts post and others respond. Among the five whose framing is known, three mostly use “crisis” and two mostly use “change”, so the central accounts are not dominated by one framing.

```
# Directed betweenness: how many accounts lie on at least one directed path between others?
bet_dir <- betweenness(g, directed = TRUE, weights = NA)
c(accounts_with_betweenness_above_0 = sum(bet_dir > 0),
  max_directed_betweenness = max(bet_dir))
```

```
## accounts_with_betweenness_above_0      max_directed_betweenness
##                                28                                5
```

Only 28 of the 1,249 accounts (2.2%) have a directed betweenness above 0, and the highest value is 5, meaning even the most “between” account lies on only 5 shortest directed paths. This confirms that directed reply chains (A replies to B, who replies to C) are almost non-existent: most replies go to accounts that never reply onwards. Directed betweenness therefore cannot identify bridges in this network, so we also measure betweenness on the undirected network.

```
# Top 10 accounts by undirected betweenness (bridges between groups)
cent |> arrange(desc(betweenness_undirected)) |> slice_head(n = 10) |>
  kable(format = "latex", booktabs = TRUE,
    caption = "Top 10 accounts by betweenness (undirected)") |>
  kable_styling(latex_options = c("scale_down", "hold_position"))
```

Two kinds of accounts act as bridges. The first are active repliers that reply to many different accounts: eliemar.bsky.social has the highest betweenness and replied to 22 different accounts, and bowprow (8), vero1986 (7) and kimnoren (4) play a similar role. By replying across several groups, they connect star-shaped groups that would otherwise be separate; in the largest component, eliemar sits at the centre and bowprow links the abdulelsayed group to the rest. The second are the heavily replied-to hubs (the did:plc:4llrhdcl... account and abdulelsayed), which join together the many accounts replying to them. All betweenness values are small (at most 0.004) because the network is so fragmented that most pairs of accounts are not connected at all.

```
# Top 10 accounts by PageRank
cent |> arrange(desc(pagerank)) |> slice_head(n = 10) |>
  kable(format = "latex", booktabs = TRUE,
    caption = "Top 10 accounts by PageRank") |>
  kable_styling(latex_options = c("scale_down", "hold_position"))
```

Table 11: Top 10 accounts by betweenness (undirected)

account	framing	in_degree	out_degree	betweenness_undirected	pagerank
eliemar.bsky.social	change	0	22	0.00393	0.0005
did:plc:4llrhdcldvdlmynkwsmg5tdc	NA	18	0	0.00351	0.0076
bowprow.bsky.social	crisis	0	8	0.00205	0.0005
abdulelsayed.bsky.social	crisis	13	0	0.00130	0.0057
vero1986.bsky.social	crisis	0	7	0.00078	0.0005
did:plc:sk6jryjhuavqchpa53jlcsyu	NA	2	0	0.00066	0.0007
kimnoreen.bsky.social	crisis	0	4	0.00056	0.0005
did:plc:uo2fna47c4v6zcnklxfhcvjb	NA	5	0	0.00045	0.0023
did:plc:iu4j537hox5huj4bwnwgub4z	NA	2	0	0.00042	0.0006
did:plc:s3m33wvdsixxr2ifnklldlfp3	NA	2	0	0.00042	0.0006

Table 12: Top 10 accounts by PageRank

account	framing	in_degree	out_degree	betweenness_undirected	pagerank
did:plc:4llrhdcldvdlmynkwsmg5tdc	NA	18	0	0.00351	0.0076
abdulelsayed.bsky.social	crisis	13	0	0.00130	0.0057
michaelhobbes.bsky.social	change	11	0	0.00007	0.0052
numbcat9.bsky.social	change	2	1	0.00000	0.0038
serverroomrelic.bsky.social	change	1	1	0.00000	0.0036
did:plc:kx375dszrbvqtu7o4xxxmojn	NA	5	0	0.00004	0.0031
whstancil.bsky.social	change	5	0	0.00002	0.0028
bug-fkr.bsky.social	change	1	1	0.00000	0.0027
dawalokyitsang.bsky.social	change	1	1	0.00000	0.0027
sheepbop.bsky.social	change	1	1	0.00000	0.0027

The top three accounts by PageRank are the same as by in-degree, confirming them as the most important accounts. PageRank also ranks some accounts with an in-degree of only 1–2 highly (e.g. numbcatt9, serverroomrelic). This is likely because they are replied to by accounts that are themselves replied to, so importance flows to them through short reply chains, which in-degree alone does not capture.

Framing and reply activity

To link the network to the overall research question, we compare how often “crisis” and “change” posts are replies to another post. We compare percentages rather than counts, so the different number of posts in each framing does not affect the comparison.

```
# Label each post as an original post or a reply
reply_rate <- raw |>
  filter(framing %in% c("crisis", "change")) |>
  mutate(post_type = ifelse(is.na(in_reply_to), "original post", "reply"))

# Share of posts in each framing that are replies
reply_rate |>
  group_by(framing) |>
  summarise(posts = n(),
    replies = sum(post_type == "reply"),
    pct_replies = round(mean(post_type == "reply") * 100, 1))

## # A tibble: 2 x 4
##   framing posts replies pct_replies
##   <chr>   <int>   <int>       <dbl>
```

```
## 1 change    1563    713    45.6
## 2 crisis     944    348    36.9
```

```
# Chi-squared test (Module 3): are framing and post type independent?
tab_reply <- table(framing = reply_rate$framing, post_type = reply_rate$post_type)
test_reply <- chisq.test(tab_reply)
round(test_reply$expected, 1) # expected counts should be at least 5
```

```
##           post_type
## framing original post reply
## change          901.5 661.5
## crisis          544.5 399.5
```

```
test_reply
```

```
##
## Pearson's Chi-squared test with Yates' continuity correction
##
## data:  tab_reply
## X-squared = 18.115, df = 1, p-value = 2.079e-05
```

H0: framing and post type (original post or reply) are independent. All expected counts are well above 5 (the smallest is 399.5). R applies a continuity correction for 2×2 tables. The test gives $\chi^2 = 18.12$ ($df = 1$), $p < 0.001$, so we reject H0. 45.6% of “change” posts are replies, compared with 36.9% of “crisis” posts. “Change” framing is therefore more common in conversation (replying to others), while “crisis” is more common in original posts. This matches 2.3 and 2.4, where “change” was associated with opinions and debate and “crisis” with news reports and events.

Limitations - Data collection and API limits: the network only includes replies found by searching for the two phrases, limited to about 2,000 posts per phrase and a single snapshot (1 Oct 2026). Replies that did not use either phrase, and earlier or later parts of conversations, were not collected. This is likely the main reason the network is so fragmented: many conversations appear only partly, as isolated pairs or stars. - **Filtering:** the network uses the raw (deduplicated) posts rather than the 2,400 cleaned posts, because the cleaning steps would have removed many replies. Framing labels are based on each account’s own collected posts, so accounts with no collected posts have no framing (grey). - **Replies only:** the network uses reply relationships only, not follows, reposts, quotes or mentions. Adding other relationships might produce a larger, better-connected network. - **Target accounts:** the replied-to account is identified from the ID in the `in_reply_to` address. Accounts that are not collected authors appear with their ID instead of a handle. Only standard account IDs (`did:plc`) are extracted. - **Construction choices:** self-replies were removed and repeated replies between the same two accounts were combined into one weighted edge, so edge counts represent unique account-to-account connections rather than individual replies.

3 Overall Findings

Answering the research questions

RQ1 – Engagement: Framing was not related to engagement ($\chi^2 = 3.40$, $df = 2$, $p = 0.183$). “Crisis” posts were only slightly more likely to receive 5 or more likes (23.1% vs 20.2%).

RQ2 – Words and topics: The framings share a common vocabulary about people, energy and global impact, but differ in emphasis. “Crisis” posts feature news, security and extreme-weather words (“nation”, “secur”, “report”, “nino”) and were about twice as likely to mention El Niño (7.7% vs 3.8%). “Change” posts feature opinion and political words (“think”, “need”, “trump”), and “hoax” appears only in the “change” word cloud.

RQ3 – Themes: k-means identified one large general-discussion cluster and four smaller themes. The framing mix differed significantly between clusters ($\chi^2 = 108.29$, $df = 4$, $p < 0.001$): “crisis” made up 78.6% of the national security cluster and 89.6% of the extreme weather cluster, compared with 37.0% overall, but only 26.4% of the opinion cluster.

RQ4 – Network: The reply network is highly fragmented (488 components; 87.3% of accounts have one connection) and organised around a few heavily replied-to accounts that do not reply back. “Change” posts were more likely to be replies than “crisis” posts (45.6% vs 36.9%; $\chi^2 = 18.12$, $df = 1$, $p < 0.001$).

What this tells us about the topic

Taken together, the results show that the choice between “climate crisis” and “climate change” on Bluesky is **not linked to how much engagement a post receives, but is linked to what the post is about and how it is used**. The null result for engagement (RQ1) led us to examine content, and three different methods then pointed in the same direction. “Climate crisis” is concentrated in original posts reporting specific events (extreme weather, El Niño death tolls, a national security report), suggesting it functions as the language of reporting urgent developments. “Climate change” is more common in replies, opinions and debate, including debate over whether climate change is real, suggesting it functions as the everyday language of discussion and disagreement. The network adds that the conversation is fragmented and largely one-way: prominent accounts post, and others respond without forming ongoing two-way discussions.

These are associations, not causes: our data cannot show that using “crisis” makes a post more news-like, or that a topic causes people to choose a particular phrase.

Limitations

- **Data collection and sampling:** the data is a single snapshot (1 Oct 2026), and two of the clusters were shaped by news stories from that time, so the themes and framing patterns may differ at other times. Bluesky users are also not representative of the wider public, so the findings describe the Bluesky conversation only.
- **API restrictions:** we collected about 2,000 posts per phrase rather than all posts, and the API occasionally returned server errors. Because only posts matching the search were collected, replies that did not use either phrase were missed. This is likely a major reason the network appears so fragmented, so it may understate how connected the real conversation is.
- **Filtering:** about 1,600 posts matched the search without using exactly one phrase in their text and were removed, and the English filter relies on users’ language tags. Posts using both phrases were excluded, so mixed framing is not studied.
- **Analytical choices:** engagement used likes only, without controlling for follower counts, and the engagement cut-offs were a judgement. The stopword list and stemming affect which words appear, the choice of $k = 5$ was a judgement with no clear elbow, and centrality describes importance within this sample only.

Collecting data over a longer period, and including follows, reposts and mentions, would show whether these framing differences hold beyond this snapshot and would give a more complete network.

GitHub Repository

All data and code are available at: All data and code are available at: <https://github.com/AakashJadhav18/comp3020-bluesky-climate-group-project>. The README explains how to reproduce the analysis from the saved data.